Python & Generative AI Beginner Guide

A compact reference document designed for learning and experimenting with a Retrieval-Augmented Generation (RAG) system.

Topics: Python basics, functions, OOP, APIs, embeddings, vector databases, LLMs, transformers, RAG, and prompt engineering.

Purpose: This is intentionally a small document. It contains enough related information to test direct questions, paraphrased questions, comparisons, and questions whose answers are not present.

1. Python Basics

Python is a high-level, general-purpose programming language known for readable syntax and a large ecosystem of libraries. Python variables do not require an explicit type declaration. The type of a variable is determined at runtime. Common built-in data structures include lists, tuples, dictionaries, and sets. A list is ordered and mutable, while a tuple is ordered but immutable. A dictionary stores data as key-value pairs. A set stores unique values. Example: `name = "Harshi"`
`skills = ["Python", "RAG", "APIs"]`
`student = {"name": "Harshi", "score": 92}` A Python program can use conditional statements such as `if`, `elif`, and `else`. Loops such as `for` and `while` are used for repetition.

Concept	Meaning
List	Ordered and mutable collection.
Tuple	Ordered and immutable collection.
Dictionary	Key-value collection.
Set	Collection of unique values.

2. Functions and Object-Oriented Programming

A function is a reusable block of code that performs a particular task. Functions can accept parameters and return values. Breaking a program into functions makes code easier to test, reuse, and maintain. Example: `def add(a, b):`
`return a + b` Object-oriented programming (OOP) organizes software around objects and classes. A class defines the structure and behavior of objects. An object is an instance of a class. Important OOP concepts include encapsulation, inheritance, polymorphism, and abstraction. Encapsulation groups data and methods together. Inheritance allows one class to reuse behavior from another class. Polymorphism allows different objects to respond to the same operation in different ways. Abstraction focuses on essential behavior while hiding unnecessary implementation details.

Concept	Meaning
Function	Reusable block of code that can accept input and return output.
Class	Blueprint that defines data and behavior for objects.
Object	An instance created from a class.

3. APIs and REST

An API, or Application Programming Interface, provides a way for software systems to communicate. A web API commonly allows a client application to send an HTTP request to a server and receive a response. REST is an architectural style commonly used for web APIs. REST APIs use HTTP methods such as GET, POST, PUT, PATCH, and DELETE. GET is generally used to retrieve data. POST is commonly used to create a resource. PUT can replace an existing resource, PATCH can partially update it, and DELETE removes a resource. A typical API response uses JSON. For example: `{"name": "Harshi", "role": "AI Developer"}` HTTP status codes communicate the result of a request. A 200 response usually indicates success, 201 indicates that a resource was created, 400 indicates a client-side request problem, 401 indicates missing or invalid authentication, 404 means a

resource was not found, and 500 indicates a server-side error.

Concept	Meaning
GET	Retrieve data
POST	Create data
PUT/PATCH	Update data
DELETE	Delete data

4. Large Language Models

A Large Language Model (LLM) is a machine learning model trained on large amounts of text to learn patterns in language. Modern LLMs can generate text, summarize documents, answer questions, classify content, and perform other language-related tasks. LLMs do not automatically know information contained in a private document that was created after their training or information that is not available to them at inference time. An application can provide additional context to an LLM through a prompt. A prompt is the input given to a model. It can contain instructions, user questions, and additional context. The model generates an output based on the information and instructions available in the prompt. LLMs can sometimes produce plausible but unsupported statements. This behavior is commonly called hallucination. Providing relevant external context is one way to reduce unsupported answers.

Concept	Meaning
LLM	A model trained to understand and generate language.
Prompt	Instructions and input provided to a model.
Hallucination	An unsupported or inaccurate generated statement.

5. Transformers

Transformers are a neural network architecture that became highly important for modern language models. A central idea in transformers is attention. Attention allows the model to consider relationships between different tokens in an input sequence. For example, when processing a sentence, attention can help the model determine which other words are relevant to the current word. Transformers process token representations using layers that include attention mechanisms and feed-forward neural networks. Models such as GPT-style language models are based on transformer architectures. A simplified flow is: `Text` → `Tokens` → `Embeddings` → `Transformer layers` → `Output probabilities` → `Generated text` Transformers are important to Generative AI because they provide a scalable architecture for learning relationships in sequences.

Concept	Meaning
Token	A unit of text processed by a language model.
Attention	Mechanism that helps a model weigh relationships between tokens.
Transformer	Neural network architecture based heavily on attention mechanisms.

6. Embeddings

An embedding is a numerical representation of information such as text. Instead of representing a sentence only as words, an embedding model converts it into a vector containing many numerical values. Semantically similar pieces of text tend to have embeddings that are close together according to an appropriate similarity measure. This makes embeddings useful for semantic search. For example, the questions "What is RAG?" and "Explain retrieval-augmented generation" use different words but may have similar meanings. Their embeddings can therefore be useful for finding related content. Embeddings are not the same thing as a vector database. An embedding model creates vectors. A vector database stores vectors and provides mechanisms for searching them efficiently. A common similarity measure is cosine similarity. It compares the orientation of two vectors rather than simply comparing their individual words.

Concept	Meaning
Embedding model	Converts information such as text into numerical vectors.
Vector	A numerical representation containing multiple values.
Semantic search	Search based on meaning rather than exact keyword matching.

7. Vector Databases

A vector database is designed to store and search vector representations. In a RAG application, document chunks can be converted into embeddings and stored in a vector database. When a user asks a question, the question is also converted into an embedding. The database compares the query vector with stored vectors and returns the most similar chunks. Examples of vector stores used in application development include Chroma, FAISS, Pinecone, and other vector database systems. For a beginner project, Chroma or FAISS can be convenient choices. The vector database does not generate the final natural-language answer. Its main role in a basic RAG system is retrieval. The retrieved text is then supplied to an LLM that generates the response.

Concept	Meaning
Storage	Stores document embeddings and associated information.
Retrieval	Finds vectors that are similar to a query vector.
Metadata	Additional information such as source name or page number.

8. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, combines information retrieval with text generation. Instead of asking an LLM to answer only from its internal learned parameters, a RAG application first retrieves relevant information from an external knowledge source. A basic RAG pipeline is: `Document → Load → Split → Embed → Store`
`User Question → Embed → Retrieve → Context + Question → LLM → Answer`
Documents are first loaded into the application. Long documents are divided into smaller chunks. Each chunk is converted into an embedding and stored in a vector store. When the user asks a question, the question is embedded and compared with stored document embeddings. The most relevant chunks are retrieved. These chunks are inserted into a prompt as context. The LLM then generates an answer using the retrieved context. RAG is useful for question-answering over private documents, manuals, policies, research material, and frequently changing knowledge. RAG and fine-tuning solve different

problems. RAG provides external information at retrieval time, while fine-tuning changes model behavior by training or adapting a model on additional examples.

Concept	Meaning
Retrieval	Finding relevant external information for a query.
Generation	Using an LLM to produce a natural-language response.
Grounding	Constraining an answer using supplied evidence or context.

9. Chunking and Retrieval

Chunking is the process of dividing a document into smaller pieces before creating embeddings. Chunk size affects how much information is represented by each vector. Very small chunks can provide precise retrieval but may lack surrounding context. Very large chunks contain more context but can also include irrelevant information. Chunk overlap repeats a small amount of content between neighboring chunks so that information near a boundary is less likely to lose context. For a beginner RAG experiment, a reasonable starting point is around 500 characters per chunk with 50 characters of overlap. This is not a universal rule. The best values depend on the document structure and retrieval method. A useful experiment is to compare chunk sizes such as 300, 500, 800, and 1000 characters while asking the same questions. Observe whether the retrieved passages become more precise or more noisy. Top-k controls how many retrieved chunks are returned. For example, k=3 means the retriever returns the three highest-ranked results.

Concept	Meaning
Chunk size	Approximate amount of text placed into one chunk.
Chunk overlap	Text repeated between neighboring chunks.
Top-k	Number of highest-ranked retrieval results returned.

10. Prompt Engineering

Prompt engineering is the practice of designing instructions and context for a language model so that it can perform a task more reliably. A RAG prompt often contains three parts: instructions, retrieved context, and the user's question. Example structure: `You are a document assistant.`
`Answer using only the provided context.`
`If the answer is not present, say that the information was not found.`

`Context:`
`{retrieved_context}`

`Question:`
`{question}` This type of instruction helps an application distinguish between the user's question and the evidence retrieved from documents. A good RAG application should be tested with questions that are directly answered by the document, paraphrased questions, comparison questions, and questions for which the document contains no answer.

Concept	Meaning

Instruction	Tells the model how to perform the task.
Context	Relevant information retrieved from external sources.
Question	The user's information request.

11. Putting the Concepts Together

Consider a PDF containing a Python and Generative AI study guide. First, a PDF loader extracts its text. A text splitter divides the text into chunks. An embedding model converts each chunk into a vector. Those vectors are stored in a vector database along with useful metadata such as the page number. A user then asks: "Why are embeddings useful in RAG?" The application converts the question into an embedding and searches the vector database. It may retrieve sections about embeddings, semantic search, and RAG. These retrieved chunks become the context for the LLM. The LLM receives the question together with the retrieved context and generates a concise answer. This separation of responsibilities is important: `Loader → reads documents`
`Splitter → creates chunks`
`Embedding model → creates vectors`
`Vector store → stores/searches vectors`
`Retriever → selects relevant chunks`
`LLM → generates the final response` Understanding these individual components is more useful than treating the entire RAG system as one black-box chain.

Concept	Meaning
Loader	Reads information from a document source.
Splitter	Divides documents into chunks.
Retriever	Selects relevant chunks for a query.
LLM	Generates the final natural-language response.

RAG Practice Questions

Use these questions after indexing this PDF. Try to inspect which chunks are retrieved before asking the LLM to generate the final answer.

1. What is the difference between a list and a tuple?

2. Why are embeddings useful for semantic search?

3. What is the difference between an embedding model and a vector database?

4. How does a transformer use attention?

5. What is the role of a retriever in RAG?

6. Explain the complete RAG pipeline from a PDF to a final answer.

7. Why is chunk overlap used?

8. What is the difference between RAG and fine-tuning?

9. How are APIs and HTTP methods related?

10. What does top-k mean in retrieval?

11. Why might very large chunks hurt retrieval quality?

12. What is hallucination and how can retrieved context help reduce unsupported answers?

13. Compare GET and POST.

14. Why are embeddings and semantic search related?

15. What is the role of the LLM after relevant chunks have been retrieved?

16. What is a convolutional neural network? (This information is intentionally NOT in the PDF.)

Suggested first RAG settings: RecursiveCharacterTextSplitter, chunk_size=500, chunk_overlap=50, top_k=3. Later, experiment with 300/30, 800/100, and 1000/150 to compare retrieval behavior.